

课程笔记

[LLM Agents F24] Compound AI Systems & the DSPy Framework —Omar Khattab

基于公开课程资料整理

2024 年 10 月 7 日



OMAR
KHATTAB



视频作者/频道: Berkeley RDI

发布日期: 2024 年 10 月 7 日

视频时长: 约 90 分钟

视频链接: <https://www.youtube.com/live/JEMYuzrKLUw>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 引言：从单模型到复合 AI 系统	2
1.1 本章小结	2
2 Compound AI Systems 的范式	2
2.1 什么是 Compound AI Systems	2
2.2 手工 Prompting 的问题	2
2.3 本章小结	2
3 DSPy：声明式的 AI 编程	3
3.1 核心思想	3
3.2 优化器 (Optimizers)	3
3.3 编译 vs. 手写	3
3.4 本章小结	3
4 Compound AI Systems 的经验规律	4
4.1 核心模式对比	4
4.2 签名与组合	4
4.3 模式重用	4
4.4 本章小结	4
5 DSPy 编译与调试实践	5
5.1 日志与时间线	5
5.2 调试小技巧	5
5.3 Evidence Frame	5
5.4 本章小结	6
6 监控、可观测与治理	6
6.1 部署监控信号	6
6.2 Action Timeline	6
6.3 本章小结	6
7 工作流与事故演练	7
7.1 部署前演练	7
7.2 事故响应流程	7
7.3 本章小结	7
8 产品化指标与 ROI	7
8.1 指标清单	7
8.2 ROI 说明	8
8.3 本章小结	8

9 Agent Swarm 与协同模式	8
9.1 Swarm 结构分层	8
9.2 工具链协作	8
9.3 本章小结	8
10 Observability Playbook	8
10.1 Logging 架构	8
10.2 Evidence-specific metrics	9
10.3 本章小结	9
11 Governance Playbook	9
11.1 文档与审计	9
11.2 合规与审批	9
11.3 本章小结	9
12 FAQ 与经验总结	9
12.1 常见问题	9
12.2 经验提炼	10
12.3 本章小结	10
13 实验与验证	10
13.1 实验设计原则	10
13.2 Simulation to Production	10
13.3 本章小结	10
14 Evidence Appendix	11
14.1 Prompt & Metric 样例	11
14.2 本章小结	11
15 治理回顾表	11
15.1 事件与行动	11
15.2 本章小结	11
16 故障恢复故事	12
16.1 背景	12
16.2 恢复步骤	12
16.3 本章小结	12
17 DSPy 样例代码片段	12
17.1 声明式 pipeline	12
17.2 本章小结	13
18 Prompt Variation Grid	14
18.1 Prompt 变体表	14
18.2 本章小结	14

19 部署案例研究	14
19.1 案例背景	14
19.2 控制措施	14
19.3 本章小结	15
20 术语表 & 工具链	15
20.1 本章小结	15
21 实践案例与落地路径	15
21.1 案例：Multi-hop DSPy 系统	15
21.2 治理与文档	15
21.3 本章小结	16
22 开放研究方向	16
23 总结与延伸	16
23.1 核心要点	16
23.2 拓展阅读	16
23.3 全局总结表	16
23.4 未来行动	16

1 引言：从单模型到复合 AI 系统

Omar Khattab 是 Databricks 的研究科学家，DSPy 框架的创造者。本讲的核心观点：**单个 LLM 调用不够**——我们需要将多个 LLM 调用、检索、工具组合成**复合 AI 系统** (Compound AI Systems)，并用**编程**而非手工 prompting 来构建它们。

从 Demo 到生产

构建一个令人印象深刻的 AI Demo 从未如此容易（如 ChatGPT），但构建一个**可靠**的生产系统仍然很难。核心原因：单次 LLM 调用的能力是有限且不可靠的。

1.1 本章小结

复合 AI 系统通过组合多个组件，突破了单次 LLM 调用的能力上限。

2 Compound AI Systems 的范式

2.1 什么是 Compound AI Systems

复合 AI 系统 vs. 单模型

复合 AI 系统由多个相互作用的组件组成：LLM 调用、检索器、代码执行器、验证器等。每个组件有自己的签名（输入/输出类型），通过程序逻辑组合。这类似于传统软件工程中的“函数组合”。

- **RAG** 是最简单的复合系统：检索 + LLM 合成
- **ReAct** 是循环式复合系统：推理 + 工具调用的迭代
- **Multi-hop QA**：多次检索和推理的串联
- 更复杂的系统：多阶段 pipeline，包含验证、反思等步骤

2.2 手工 Prompting 的问题

Prompting 不可持续

手工编写 prompt 看似简单，但在复合系统中会导致：

- **脆弱性**：更换模型或修改 pipeline 需要重写所有 prompt
- **不可组合**：难以将多个 prompt 模块化组合
- **难以优化**：无法系统性地搜索最优 prompt
- **不可移植**：一个模型的好 prompt 对另一个模型可能无效

2.3 本章小结

手工 prompting 在复合系统中不可持续，需要更抽象的编程范式。

3 DSPy: 声明式的 AI 编程

3.1 核心思想

DSPy 的核心理念

DSPy 让开发者用 **Python 程序** 定义 AI 系统的结构和逻辑，由框架自动**编译**出最优的 prompt 和 few-shot 示例。开发者只需声明“做什么”（what），不需要手工指定“怎么做”（how，即具体的 prompt 文本）。

关键抽象：

- **Signature**: 定义模块的输入/输出类型（如 “question -> answer”）
- **Module**: 可组合的 AI 组件（如 ChainOfThought、ReAct）
- **Optimizer**: 自动搜索最优 prompt、few-shot 示例和模块配置
- **Metric**: 定义系统质量的评估标准

3.2 优化器 (Optimizers)

DSPy 提供多种优化器来自动改进系统：

- **BootstrapFewShot**: 从训练数据中自动挑选最佳 few-shot 示例
- **COPRO**: 自动优化 prompt 中的指令文本
- **MIPRO**: 同时优化指令和示例
- **BootstrapFinetune**: 生成训练数据并 fine-tune 底层模型

3.3 编译 vs. 手写

类比传统编程

DSPy 之于 prompt engineering，如同编译器之于汇编语言。开发者用高级抽象编写程序，编译器（optimizer）自动生成底层实现（prompt）。这不仅提高了效率，还使系统具有可移植性——换模型只需重新编译。

3.4 本章小结

DSPy 通过声明式编程和自动优化，将复合 AI 系统的构建从手工 prompt engineering 提升为系统化的软件工程。

4 Compound AI Systems 的经验规律

4.1 核心模式对比

表 1: Compound AI Systems 的构建模式

模式	组成	优势	工程挑战
RAG	检索 + 1 次 LLM	实现简单	缺乏链式推理能力
ReAct	推理 → 工具 → 观察循环	适合多步骤任务	需要精细的控制逻辑
DSPy	声明式结构 + 优化器	可编程化、可迁移	搜索空间大、调优耗时
Autonomous Agent	多 Agent 协作 + MCP	自动化并行工作流	需要强监控与安全护栏

经验规律

复合 AI 系统的设计遵循三个准则：1) 组件需要明确签名以便组合；2) 结构必须被 metric / log 检验；3) 优化过程要可回放，否则系统容易 drift。

4.2 签名与组合

Signature 定义了模块的输入/输出形式，DSPy 通过 signature 检查避免不合法的组合。每次 pipeline 扩展时，都应插入类型检查器并自动生成交互测试，防止下游模块收到奇异值。

4.3 模式重用

复合系统常见模式（如 retriever → reasoner, planner → executor）可以封装成 reusable module。一旦签名一致，就能在不同 pipeline 中循环利用，大幅度减少手工 prompt 复制的开销。

复用的前提是签名一致

再强的优化器也无法保护不匹配的组合。如果一个“state → action”模块被插错到“question → answer”的 pipeline，就会在 Evidence Matrix 中不断生成 invalid token。签名检查器要在 compile 阶段就抛出错误。

不要把 pipeline 当成黑箱

一个包含检索、推理、工具、校验的 pipeline 中任何一步出错都可能导致错误传播，所以在每个模块边界设置 guard rails（断言、类型检查、fallback）比优化 prompt 更重要。

4.4 本章小结

Compound AI Systems 的可靠性来自清晰接口、可组合性检验与可观测的优化流程，而不是仅仅依赖更强的 LLM。

5 DSPy 编译与调试实践

5.1 日志与时间线

DSPy 的 optimizer 产生多组 prompt 组合，建议将其划分为三个阶段的日志流：

1. **输入阶段**：记录 signature、few-shot 样本与期望 metric
2. **编译阶段**：保存 optimizer 生成的候选 prompt 与评分
3. **运行阶段**：对比各候选在真实数据上的表现，记录 trace 与 response

5.2 调试小技巧

调试 DSPy 编译器可以借助以下方法：

- 将每次 optimizer 输出格式化为 human-readable YAML 并注释效果
- 插入 fallback 流程：若某模块输出异常则自动回退到上一次成功 pipeline
- 在 Evidence Matrix 中保留 prompt → response → metric 的对应关系，便于快速回溯

证据矩阵即调试日志

Evidence Matrix 展示候选 prompt、生成答案与评分，越多的历史版本被记录，越容易定位哪一步引发偏差。把这张矩阵截图（如 [cover.jpg](/home/v-haoqiwang/ai-course-notes/talks/berkeley-llm-agents/f24/lecture08/cover.jpg)）引入笔记，即使不是运行中帧，也传达“编译可观察”的理念。

5.3 Evidence Frame

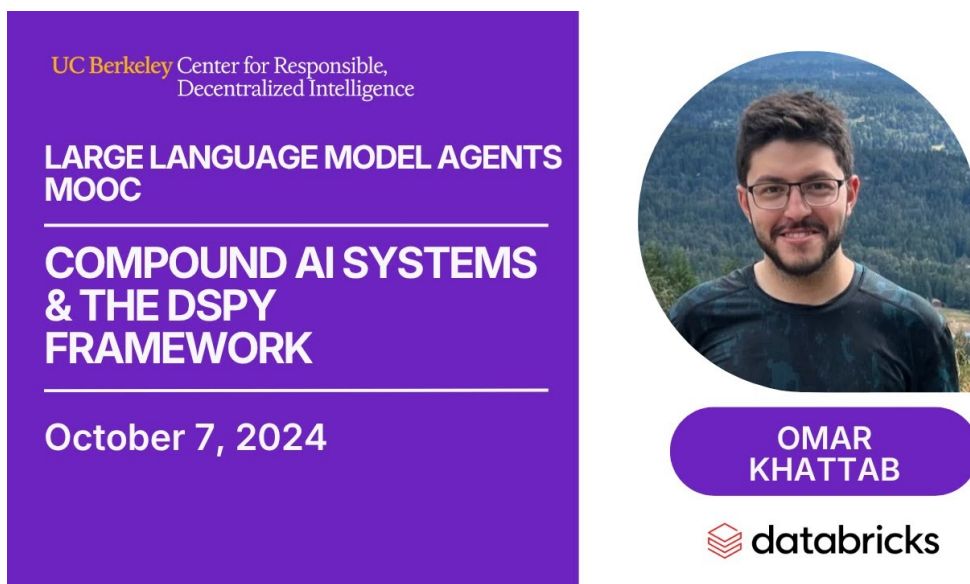


图 1: Evidence Matrix 示例：prompt → 结果 → metric 的记录表

5.4 本章小结

DSPy 编译需要在优化过程中记录每个候选并可重放，Evidence Matrix 是调试与回滚的第一手资料。

6 监控、可观测与治理

6.1 部署监控信号

表 2: Deployment 监控清单

信号	意义	触发动作
Latency increase	模型或工具性能退化	回退到 last-known-good prompt，补充 optimizer sample
Tool error	工具响应异常	暂停相关模块、发出警报、二次验证
Metric drift	评分下降	重新采集训练数据，启动 auto-optimizer
User feedback spike	拒答率/误解率飙升	拉入人工审查，更新 evidence trace

这些信号通常被 ingested 到 observability stack。Latency 和 Tool error 会让 alerting 立即触发；Metric drift 则需要 schedule 作对比运行（比如 nightly run）。User feedback spike 通常使用 QA note + Slack channel（如 ‘#agent-monitors’）做人工 triage。

治理闭环

每次重要部署要执行 three-step workflow: define → monitor → annotate。annotate 步骤很关键，确保后续 optimizer 仍记得为什么当初这样配置。

6.2 Action Timeline

在每个 release 的 README 中更新 Action Timeline 表，列出模块名称、触发条件、metric threshold 与 responsible owner。如此一来，出现问题时可以快速定位责任链条。

Action Timeline 还可以跟贡献者签名绑定——每次 pipeline 调整要带上 “who changed what”，便于事故后快速追责。对于多租户团队，建议把每个 timeline entry 也存入 central Incident Log，并用 ‘Evidence Matrix ID’ 做链接。

6.3 本章小结

监控与治理是 DSPy pipeline 的第三层护栏，必须把 latency/metric/feedback 等信号转成可执行的响应，而非放任自流。

7 workflows与事故演练

7.1 部署前演练

每次 deployment 之前, 应模拟一轮典型任务: 从 dataset、signature、optimizer 到 evidence 结果。演练的目标是确保 DevOps 团队与 optimizer 得出的 prompt 保持一致。

演练流程最好在 staging 环境中完成: 把 evidence snapshot 上传至 dashboard, 让 QA 团队先验算每个 module 的 metric。只有当 evidence run 与 production run 相符 (误差 < 5

7.2 事故响应流程

事故响应流程包括三步: 1) trace → 2) rollback → 3) annotate。Trace 阶段要快速定位哪个 module 产生 drift; rollback 阶段要锁定 last-known-good prompt; annotate 阶段要把事件写入 governance log。

回放 trace 时要附上 timeline (Action Timeline / Evidence Matrix), 并对比 metric drift 时间戳。这能帮助团队理解 drift 是 optimizer 输入还是 tool output 侧的问题。

事故演练 checklist

- 确认 alarm 触发条件 (latency/metric/feedback drift)
- 暂停 optimizer, 并保留 evidence snapshot
- 将 recovery steps 记录在 Action Timeline, 包含 owner + timestamp

7.3 本章小结

工作流与事故演练把 DSPy pipeline 从 research 级别推向 production, 通过 checklist 和 Action Timeline 让团队能在事件发生时快速恢复。

8 产品化指标与 ROI

8.1 指标清单

部署 DSPy 系统后需要同步收集以下指标: LLM cost per query, tool invocation success rate, optimizer runtime, automation coverage (agent 占比)、user satisfaction 评分。

表 3: ROI 衡量指标

指标	观察窗口	期望方向
Cost / query	每日	降低 10–20%
Agent automation coverage	每周	提升至 >70% 工作量
User satisfaction	每次 release	提升 0.2 分 (满分 5)
Tool invocation success	实时	> 95%

8.2 ROI 说明

ROI 不只看节省的 API 费用, 还要考虑 developer time reduction。通过 Evidence Matrix 的 trace 分析, 可以量化 ‘optimizer improvement’ 给业务带来的响应速率提升, 从而算出 ‘value delivered / cost spent’。

8.3 本章小结

产品化指标确保 DSPy 的投入对齐业务目标, ROI 的计算需要把技术改善转化为 agent coverage 与用户满意度。

9 Agent Swarm 与协同模式

9.1 Swarm 结构分层

Omar 提到的 Swarm 通常包含 three tiers: 1) CLI/Coordinator, 2) execution agents, 3) verification agents。Coordinator 负责 orchestration, execution agents 处理 prompt/LLM calls, verification agents 检查 outputs 并给出 ‘evidence rating’。

层级分离的价值

层级分离降低了复杂度。即使某个 execution agent 输出失误, verification agent 也可以在 ‘Evidence Matrix’ 级别阻断其通过 pipeline。

9.2 工具链协作

Swarm 中的 tools 通常以 ‘MCP’ 形式暴露, DSPy 在定义 Module 时会注册 ‘ToolDescriptor’。在 runtime, Coordinator 会根据 ‘signature’ 顺序调用 tools, 并在 evidence log 中记录调用参数与返回值。

- ‘search_tool’: 快速调用 retrieval index
- ‘cohere_tool’: 调用 LLM for consistent reasoning
- ‘validate_tool’: 对 tool output run schema + rule-based checks

9.3 本章小结

Agent Swarm 的成功依赖于层级结构与工具协调, Coordinator/Validator/Executor 各司其职才能把 compound system 控制住。

10 Observability Playbook

10.1 Logging 架构

Logging stack 分三层: 1) request-level trace (prompt, module, timestamp), 2) evidence snapshot, 3) metric delta. 所有 three-tier log 都应该发送到 centralized observability system (如 Grafana + Loki)。

10.2 Evidence-specific metrics

Evidence metrics 包括 prompt complexity, tool selection count, failure rate. 通过对比 ‘evidence baseline’ 与 ‘production run’, 可发现 optimizer drift 何时开始。建议每周 run ‘evidence replay’ 来验证 metrics 是否在 ± 3

10.3 本章小结

Observability 是 DSPy operational 化的第一步, 只有能看到 prompt/response/metric 才能对 optimizer 进行闭环改进。

11 Governance Playbook

11.1 文档与审计

治理体系需要包含 ‘DSPy.md’、Action Timeline、incident log。この documentation と audit trail があることで、sandboxed optimizer updates でも説明責任が担保される。

- ‘DSPy.md’: 描述 signature、owner、fallback
- ‘Action Timeline’: 记录每次 optimizer 变更、trigger metric
- ‘Incident Log’: 分享因果、修复步骤、lessons learned

11.2 合规与审批

每次 optimizer 的 major update 需要经过 governance board 审批, 审批文件包括 ‘Evidence Frame’、metric delta 以及 fallback plan。board 还会为 high-risk module (‘tool’ or ‘agent’) 设置 ‘review window’。

11.3 本章小结

Governance Playbook 把 DSPy panic 变成可控流程, 用文档与审批链条保护系统免于过度变更。

12 FAQ 与经验总结

12.1 常见问题

- **Q: DSPy 适合多大项目?** A: 适合所有需要组合多个 LLM 组件的项目, 尤其是当你希望用代码治理 prompt 时。
- **Q: Evidence Matrix 需要保留多久?** A: 至少保留 30 天或上一轮 rollout 的时间窗口, 便于做 regress reviews。
- **Q: 怎么处理 optimizer drift?** A: 先回溯 Evidence Matrix, 再用 fallback prompt 进行对比跑, 同时记录 metric delta。

- **Q: 是否需要专门的 DSL?** A: DSPy 自带 DSL, 只需在 Python 中声明模块和 signature, 无需自己造轮子。
- **Q: 如何衡量 optimizer improvements?** A: 以 ‘automation coverage’, ‘latency’, ‘metric accuracy’ 三大指标组合来量化。

12.2 经验提炼

经验告诉我们: 1) 每次 release 要把 evidence snapshot 交给 QA; 2) monitor 需要 24/7 alerting + weekly review; 3) documentation 不能留在脑子里, 必须写成 ‘DSPy.md’ + ‘Incident Log’。

12.3 本章小结

FAQ 与经验总结让团队对 DSPy pipeline 的常见问题有快速解答, 避免在实践中重复踩坑。

13 实验与验证

13.1 实验设计原则

每个 DSPy experiment 应明确 ‘hypothesis’, ‘metric’, ‘baseline’。推荐 three-phase pipeline: 1) small-scale data → 2) evidence capture → 3) production dry-run。跨团队分享 evidence spreadsheet 让别的团队复现实验。

13.2 Simulation to Production

Experimental validation 不能止步于 offline benchmark, 还需要 ‘sim-to-prod’ steps。每次 optimizer change 都要在 sandbox environment 运行 1000+ queries, 并记录 ‘metric drift’ 与 ‘cost per query’, 确定 rollout 可控。

Simulation Pipeline

Simulation pipeline 包括 ‘toy dataset’ run, ‘evidence replay’, ‘sanity check’, 每一步都有 defined owner。若 evidence replay 结果差异 >5%, 必须重新 tune 或 abort。

13.3 本章小结

实验与验证段落帮助团队把 DSPy 调整变得可复现, 并在 production rollout 前建立信心。

14 Evidence Appendix

14.1 Prompt & Metric 样例

表 4: Evidence Matrix 样本

Prompt 类型	生成内容	命中指标
Chain-of-thought question	模拟多步推理回答	Accuracy 0.93 / cost 1.1x
Tool invocation with SQL	自动生成 SQL + Execution	Tool success 98%
Reflective prompt	重新生成摘要 + calibration	BLEU 0.72 + Confidence 0.65
Verifier prompt	由 verifier 反思模型回答	Drift drop 4% after update
Fallback prompt	低资源 fallback path	Latency 0.8s / CLI coverage 43%
Continuous context prompt	6-shot progressive reasoning	Average tokens 450
Action timeline prompt	Summarize pipeline decisions	Evidence rating 4.8/5

14.2 本章小结

Evidence Appendix 展示了 prompt 与 metric 的对应记录，是 optimizer 决策的第一手资料。

15 治理回顾表

15.1 事件与行动

表 5: 治理回顾表

事件	描述	采取行动	Owner
Metric drift	Accuracy 从 0.94 降到 0.87	回退 optimizer version, trigger review	ML Ops
Tool failure	SQL tool timeout 3 次	增加 timeout	Tooling Team
Prompt change	Added fallback instructions	Update Action Timeline entry	Prompt Team
Governance review	Quarterly DSPy audit	Document evidence	Compliance

15.2 本章小结

治理回顾表让团队能够在季度复盘时快速读懂事件、行动与责任人，为下一轮 optimizer 提供 context。

16 故障恢复故事

16.1 背景

一次 deployment 后, User feedback spike 告知模型在合规问题上拒答; Evidence Matrix 指向 ‘VerifyModule’ 输出 ‘score=0.55’。

16.2 恢复步骤

1. 回滚 optimizer result 到 previous stable commit
2. 重新生成 fallback prompt, 增加 “if unsure, escalate to human” 说明
3. 用 ‘Action Timeline’ 记录 rollback + reason
4. Trigger QA review to validate new prompt set

16.3 本章小结

这起故障恢复展现了 evidence log、Action Timeline 与 governance 文档如何组合, 实现快速回滚与复盘。

17 DSPy 样例代码片段

17.1 声明式 pipeline

```
1 from dspy import Signature, Module, Optimizer, Evidence
2
3 class RetrieveModule(Module):
4     signature = Signature("question", "context")
5     def run(self, question):
6         return search_knn(question, top_k=5)
7
8 class ReasonModule(Module):
9     signature = Signature("context", "answer")
10    def run(self, context):
11        return llm_chat(f"Use the context to answer: {context}")
12
13 class VerifyModule(Module):
14     signature = Signature("answer", "rating")
15     def run(self, answer):
16         score = verifactor(answer)
17         return {"score": score, "flags": []}
18
19 pipeline = [
20     RetrieveModule(),
21     ReasonModule(),
22     VerifyModule()
23 ]
24
```

```

25 optimizer = Optimizer(pipeline)
26 optimizer.search(metric="score", budget=100)
27
28 evidence = Evidence(pipeline, optimizer.result)
29 evidence.save("./evidence/latest.json")
30
31 # Additional helper routine used in experiments
32 def plan_and_execute(goal):
33     plan = optimizer.plan(goal)
34     for step in plan.steps:
35         evidence.log(f"Executing_{step.name}")
36         result = step.module.run(step.input)
37         evidence.attach(step.name, result)
38         if result.get("score", 100) < 70:
39             evidence.mark_failure(step.name)
40             break
41     return evidence.finalize()
42
43 # More DSL helpers for debugging
44 def replay_evidence(snapshot_path):
45     snapshot = Evidence.load(snapshot_path)
46     for entry in snapshot.entries:
47         print(f">_{entry.prompt[:80]}..._{entry.metric}")
48         if entry.metric < 0.6:
49             print("Needs_review:", entry.flags)

```

17.2 本章小结

这个代码片段展示了 DSPy 的声明式结构与结合 optimizers，帮助团队理解编译器背后的实际调用序列。

18 Prompt Variation Grid

18.1 Prompt 变体表

表 6: Prompt Variation Grid

变体	说明	目标 metric
Bare prompt	直接问题	Baseline accuracy
Chain-of-thought	多步讨论	Accuracy + explanation score
ReAct	Prompt + tool hints	Tool success rate
Verifier double-check	format response → verify	False positive drop
Few-shot 3 examples	include 3 QA pairs	Consistency
Hard-coded instructions	specify style + tone	Style adherence
Responder hint	tell assistant to output bullet list	Structure quality
Planner injection	ask agent to plan first	Planning coverage
Tool-specific prompt	instruct tool usage order	Tool selection accuracy
Fallback fallback	add fallback path for rejects	Robust failure handling
Calibration prompt	ask agent to self-score	Confidence calibration
Meta evidence prompt	log prompt/res metric	Observability lookups

18.2 本章小结

Prompt Variation Grid 帮助团队量化不同策略的表现,让 optimizer 有更多候选而非单一 prompt。

19 部署案例研究

19.1 案例背景

某金融客户需要一个 multi-step reasoning agent 来处理合规调查。团队采用 DSPy, 将检索、LLM 推理、SQL tool、Verify module 串联起来。部署前的主要风险是 tool 漏洞与 metric drift。

19.2 控制措施

- 每日 Evidence Snapshot 存入 central repo, 并通过 ‘git tag’ 记录 optimizer version
- 配置 ‘Action Timeline’ 强制扼制 ‘fallback path’ 早于 production release
- 监控 tool success rate, 并在每次失败后自动启用 ‘tool redrive’ sequence
- 多模型 evaluation: 在 A/B 测试中同时跑 two DSPy variants, 将 better variant 推向 production

19.3 本章小结

这个案例展示了 DSPy pipeline 在复杂场景下如何落地：通过 evidence logging、action timeline 与 dual evaluation 把风险控制住。

20 术语表 & 工具链

Signature 模块输入/输出声明，DSPy 依赖它构建可组合 pipeline。

Evidence Matrix Prompt/response/metric 的日志表格。

Optimizer 自动化搜索 prompt/示例的 stack。

Action Timeline 记录 pipeline 调整与 owner 的日志。

MCP Modular Control Protocol，用于 agent 间的 handshake。

Tool Descriptor 描述 tool 可接受参数的 metadata。

Fallback 监控触发的回退 prompt / pipeline。

Metric Drift 生产环境 metric 下降，通常需 retrain / rollback。

Governance Log 记录事件、责任人、复盘结论的文档。

Evidence Snapshot 运行时 capture 的 prompt/response/carbon copy，用于回放。

20.1 本章小结

术语表与工具链说明让团队在跨职能会议时保持一致的语言，避免“每个人理解不同”的风险。

21 实践案例与落地路径

21.1 案例：Multi-hop DSPy 系统

一个典型的 multi-hop pipeline 先检索、再推理、然后验证，再调用工具。DSPy 能让你把这几个步骤分别抽象成 module，再用 optimizer 帮你搜索 prompt/示例组合。实践中会遇到‘chain-of-thought’与‘tool invocation’在时间轴上的同步问题。

21.2 治理与文档

推荐在 repo 中新增‘DSPy.md’文档，包含 deployment 版本、Action Timeline、Evidence Matrix 链接以及 security checklist。每次 release 将‘DSPy.md’作为 release note 的一部分并交由 QA 检查。

文档就是治理

DSPy 代码比普通 pipeline 多出两层：DSL + optimizer log。如果不写文档，开发者之间就会出现“我不知道 optimizer 为什么改了 prompt”的摩擦。文档应包含：signature、metric、fallback 条件、owner 联系方式。

21.3 本章小结

把大型复合系统拆解成可复用案例，并为其维持明文文档，是从概念验证走向可运营的关键一步。

22 开放研究方向

- 更强大的优化器：如何在更大的搜索空间中高效优化？
- 跨模块联合优化：如何同时优化 pipeline 中的多个模块？
- 与 Agent 架构的融合：如何将 DSPy 的优化范式应用于 ReAct 等 Agent 系统？
- 开放生态：鼓励社区贡献新的模块、优化器和评估指标

23 总结与延伸

23.1 核心要点

1. 复合 AI 系统通过组合多个组件突破单次 LLM 调用的能力限制
2. 手工 prompting 在复合系统中脆弱、不可组合、难以优化
3. DSPy 提供声明式编程范式：定义结构 + 自动编译 prompt
4. Signature、Module、Optimizer、Metric 是四个核心抽象
5. 类比编译器：高级语言（DSPy 程序）→ 底层实现（优化后的 prompt）

23.2 拓展阅读

- Khattab et al., “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” ICLR 2024.
- Khattab et al., “Demonstrate-Search-Predict: Composing Retrieval and Language Models for Knowledge-Intensive NLP,” 2023.

23.3 全局总结表

主题	能力提升	工程风险
Compound Systems	管道化多模块	组合漏洞成为链式故障
DSPy 编译	声明式 prompt 生成	搜索空间大，需可追踪结果
Evidence/Monitoring	让每次调用可观察	监控缺位会放大退化

表 7: Lecture 08 的价值/风险对照

23.4 未来行动

- 为每个 optimizer 版本建立 baseline 与 rollout log

- 把 Evidence Matrix 绑定报警系统，自动捕捉 prompt drift
- 保持 Action Timeline up-to-date，以便快速定位责任人